



Cyber序列化方式



时药

关注

1 人赞同了该文章 >

Apollo中Cyber的[序列化](#)方式是通过ProtoBuf的方式来完成序列化和反序列化的，即在各模块中，Apollo消息传递的格式定义是通过pb文件来定义的。

protobuf是Google开源的序列化工具，支持一种[跨语言](#)、跨平台、可扩展的用于系列化数据协议。

protobuf中最基本的[数据单元](#)是message，并且在message中可以多层嵌套message或其他的基础数据类型的成员。

Cyber支持的RTPS协议进行通信的情况下，实际传输的格式是通过CDR序列化后进行传输的。对应于Cyber中的UnderlayMessage的封装。

UnderlayMessage有4个私有成员

```
private:
    int32_t m_timestamp;
    int32_t m_seq;
    std::string m_data;
    std::string m_data_type;
```

Protobuf序列化后，会将序列化后的数据传递到UnderlayMessage的m_data当中。

```
UnderlayMessage m; RETURN_VAL_IF(!message::SerializeToString(msg, &m.data()), 1
```

UnderlayMessage实现了CDR中对序列化数据所需要的方法。这样在fast-rtps中可以直接使用UnderlayMessage实现的方法和填充包裹的数据（以及UnderlayMessageType）。

如果简单和Fast-RTPS来对应：

UnderLayMessage =》 Data

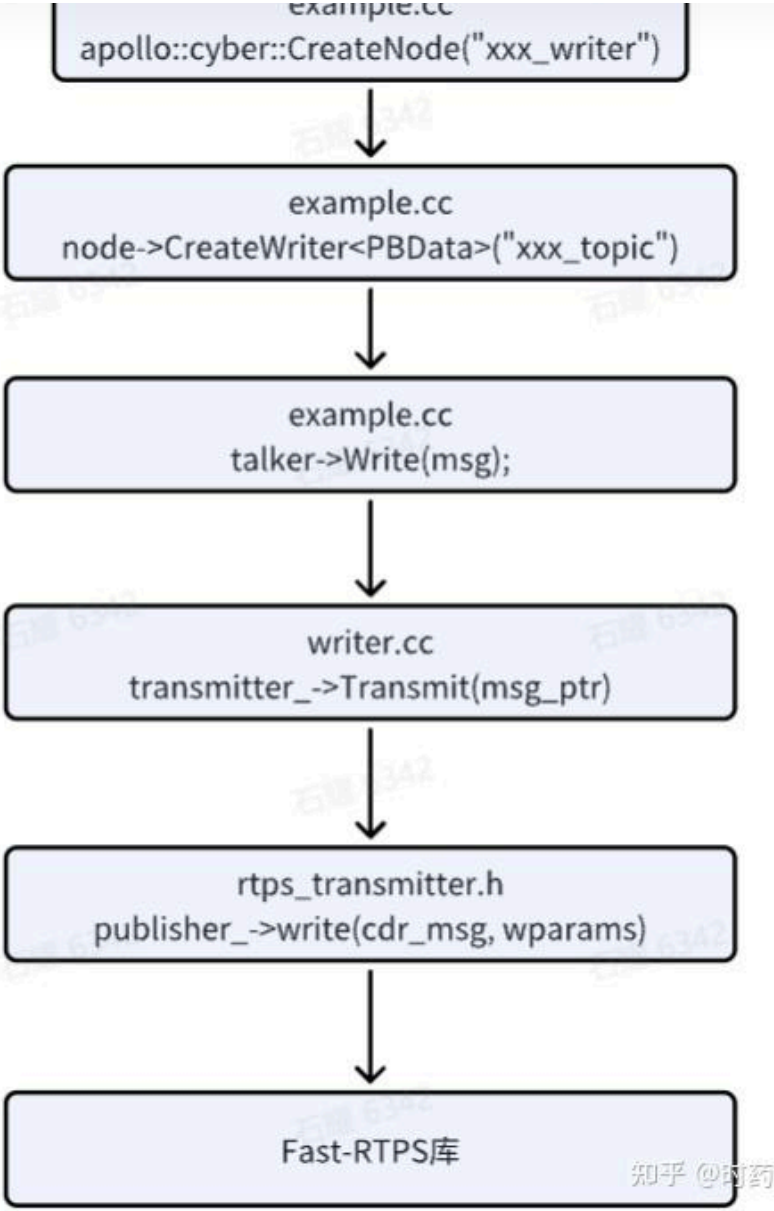
UnderlayMessageType =》 DataPubSubType

相对于[Fast-DDS](#)使用Fast-DDS-gen来生成了对应的Data和DataPubSubType的[头文件](#)，cyber直接通过自己实现的UnderlayMessage和UnderlayMessageType来封装了protobuf序列化的字符串。

protobuf——》CDR这个步骤实际上是可以简化的。可以使用CDR和IDL的这种Fast-DDS原生的序列化方式来对数据进行定义，不一定要[protobuf](#)来插一脚。

当然使用protobuf也是有好处的，因为互联网中很多基础架构也都是支持protobuf的格式，所以基于protobuf去做一些扩展功能会比较方便一些，而Fast-DDS中的CDR方式则不像前者使用范围那么广，所以如果使用了CDR方式去做序列化，当做一些扩展的时候，可能就会涉及到协议转换了（CDR-》protobuf）。





编辑于 2023-09-13 15:24 · IP 属地北京

cyber C++ 序列化

▲ 赞同 1 ▼ ● 添加评论 ↗ 分享 ❤ 喜欢 ★ 收藏 📄 申请转载 ...

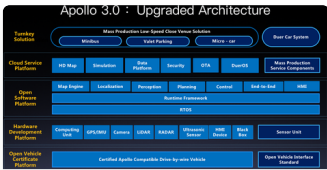


理性发言，友善互动



还没有评论，发表第一个评论吧

推荐阅读



apollo介绍之Cyber框架(十)

王方浩 发表于自动驾驶



Apollo6.0学习003: Cyber RT 常用命令

Andre



apollo介绍之cyber启动(十九)

王方浩 发表于自动驾驶



Cyber python版本

王方浩 发表于